Kaagazy Technologies Private Limited
CIN - U63999UT2025PTC020308
20 Dav College Road, Karanpur,
Dehradun - 248001, Uttarakhand, India

SWASTHIQ · SDE INTERN HIRING — JULY 2026

EOD Billing & Analytics Agent

A take-home assignment for the SDE Intern screening process.

Overview

- **The Context:** SwasthiQ's Kaagazy platform runs on real clinic billing data every day. This assignment mirrors that domain on a synthetic dataset — no production data or code is involved.
- **The Task:** Build a Python REST API that ingests a clinic's daily billing log and produces (1) a deterministic end-of-day reconciliation report, (2) basic analytics, and (3) an LLM-generated narrative summary of both — plus a React frontend that presents all three as shown in the UI Requirements section.
- **What We're Evaluating:** Core backend engineering discipline, and how carefully you keep an LLM grounded in real numbers — not how well you can prompt one.
- **Time Budget:** 3 days from the date this assignment is shared with you. Late submissions will not be considered. Prioritize a smaller scope done rigorously over everything done loosely.

The Scenario

A clinic's front desk logs every transaction as the day happens: consultations, medicine sales, refunds, partial payments.

At the end of the day, the clinic owner needs four things, fast, without opening a spreadsheet:

- **1.** How much was billed vs. how much was actually collected
- **2.** Which hour of the day did the most business
- **3.** Which medicines moved the most — by quantity, and separately, by revenue
- **4.** A plain-language summary they can read on WhatsApp

You are building the service that produces all four, on demand, from a raw billing log.

Input: Billing Log Schema

Field	Type	Notes
clinic_id	string	Single clinic per file
visit_id	string	Unique per visit
timestamp	ISO 8601, UTC	Used for hour-of-day bucketing
doctor_id	string	Not required for this assignment's outputs
line_items	array	{ drug_name, qty, unit_price_paise }
payment_mode	enum	cash / card / upi
amount_paid_paise	integer	Amounts are integer paise, not float rupees — intentional
discount_paise	integer	May be 0
is_refund	boolean	If true, amount_paid_paise is a negative adjustment

A sample dataset (3 clinic-days, including edge cases) is provided alongside this brief.

Core Requirements — Deterministic Layer

- ☐ Parse and validate the log; reject malformed rows with a specific, actionable error — not a generic 500.
- ☐ Compute the EOD reconciliation: total billed, total collected, outstanding, and refunds — each split by payment mode.
- ☐ Compute analytics: revenue by hour-of-day, top medicines by quantity, and top medicines by revenue — as two distinct rankings.
- ☐ This layer must never call an LLM. It is your ground truth, and everything downstream is checked against it.
- ☐ Store money as integer paise throughout. Tests should cover at least one non-happy-path day.

Kaagazy Technologies Private Limited
CIN - U63999UT2025PTC020308
20 Dav College Road, Karanpur,
Dehradun - 248001, Uttarakhand, India

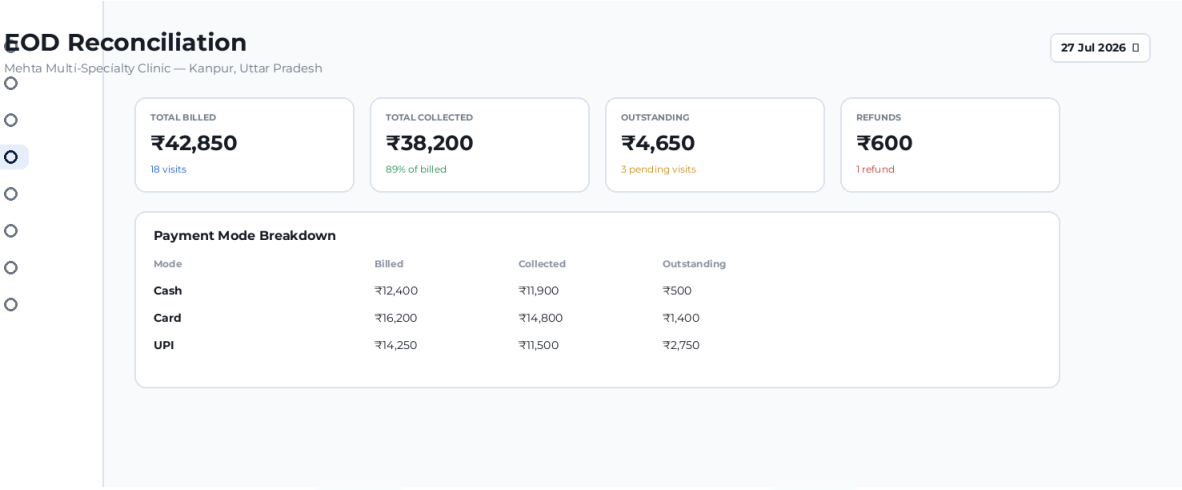
Agentic Requirements — Narrative Layer

- ☐ Given the deterministic report as input, generate a short, owner-facing summary in a WhatsApp-appropriate tone.
- ☐ Every figure that appears in the narrative must trace back to the report. Zero invented numbers — this is graded automatically.
- ☐ If a metric can't be computed from the data provided (for example, profit, when cost price isn't given), say so plainly. Do not approximate it as something else and present it as fact.
- ☐ Handle a malformed or off-schema model response — it should not silently corrupt your output or crash the request.

UI Requirements — Build This Exact Interface

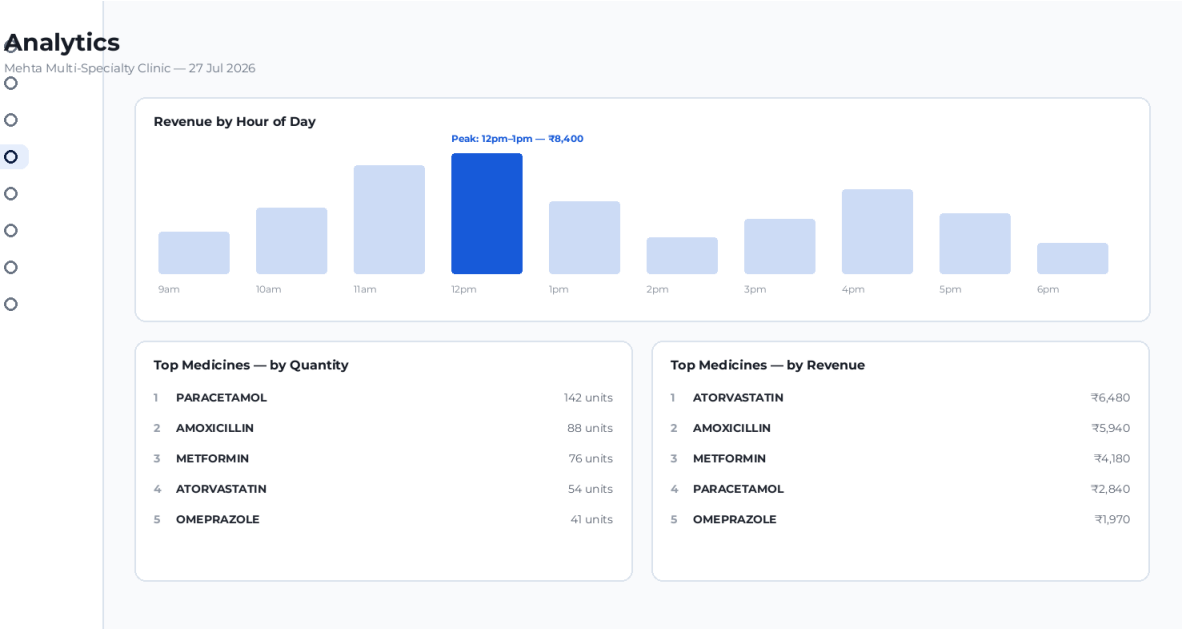
Your backend is only half the assignment. The three screens below are the actual interface for this feature — build your React frontend to match them, not a rough approximation. Structure, content, and layout should match; pixel-perfect visual polish is not the point of this exercise.

1. EOD Reconciliation Dashboard



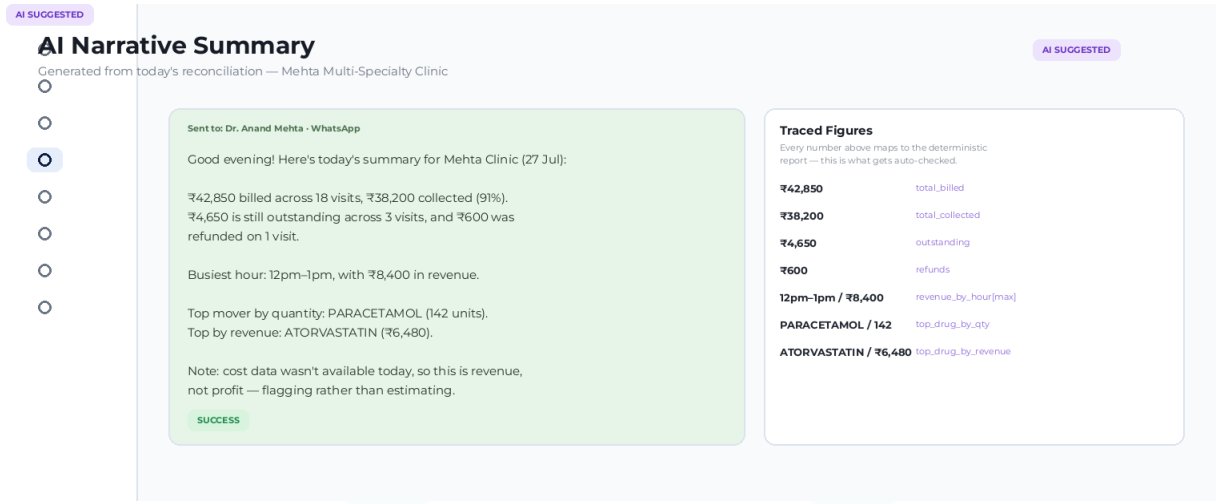
Stat cards for billed / collected / outstanding / refunds, plus a payment-mode breakdown table.

2. Analytics



Revenue-by-hour chart with the peak hour called out, and two separate rankings — by quantity and by revenue. Keep these as two distinct lists.

3. AI Narrative Summary



The generated narrative on the left; the "Traced Figures" panel on the right maps every number in it back to the report field it came from — this is the visual proof of the grounding requirement above.

A shared sidebar (as shown) should persist across all three screens.

Kaagazy Technologies Private Limited
CIN - U63999UT2025PTC020308
20 Dav College Road, Karanpur,
Dehradun - 248001, Uttarakhand, India

Constraints

- Any LLM API is fine — use your own key, or request a capped shared test key from us.
- SQLite or in-memory storage only. No managed Postgres/AWS setup required — we're evaluating pipeline and API design, not infra plumbing.
- Backend: Python, exposed as a REST API. Frontend: React. This matches the submission structure below and is not optional for this assignment.

Submission Guidelines

- **1. Single Repository:** Provide a single public GitHub repository containing the complete project. The repository must clearly separate:
- **/backend —** Python REST API implementation.
- **/frontend —** React application.
- **README.md —** Documentation and API contracts.
- **2. Live Link:** A working, publicly hosted link (e.g., Vercel, Netlify) to the application.
- **3. Technical Explanation:** A brief README explaining the REST API structure you designed for the functions, and how your Python functions ensure data consistency upon update.
- **4. Deadline:** The assignment is to be completed in 3 days from the date it is shared. Late submissions will not be considered.

How This Gets Evaluated

- Correctness of the deterministic reconciliation and analytics layer.
- Whether every number in the narrative matches the underlying report exactly.
- Whether the three screens match the structure and content shown above — not just a functional API with no UI.
- Code structure, error handling, and test coverage.
- How edge cases in the data are handled — we've included a few in the sample dataset on purpose.

- Shortlisted candidates will have a short live follow-up where we extend the assignment together in real time.

Good luck. Build the intelligence layer.